

GGPL Quote — On-prem (office LAN) deployment

Run the whole portal on one office PC ("the server"). Colleagues use it from their own computers over the office network. Nothing is deployed to the public internet.

The stack: **Postgres** (database) + **API** (FastAPI) + **Web** (Next.js), all in Docker containers. Only the web port (3000) is exposed to the LAN; the database and API stay on the internal Docker network.

A. Recommended: Docker (one command)

1. Prerequisites (on the server PC only)

- Windows 10/11 (or any OS) with **Docker Desktop** installed and running.
- The project folder copied onto the machine (any location works — the data lives in a Docker volume, not in the project folder, so OneDrive is fine).

2. Configure secrets — create `.env`

Copy `.env.example` to `.env` and set real values:

```
AUTH_SECRET=<long random string>      # e.g. python -c "import secrets; print(secrets.token_urlsafe(32))"
POSTGRES_PASSWORD=<a strong password>
OPENAI_API_KEY=sk-...                  # for enquiry text extraction
```

The current `.env` contains a real OpenAI key that has been visible in chat — **rotate it** (create a new key, paste it here, revoke the old one).

3. Start it

From the project root:

```
docker compose -f docker-compose.prod.yml up -d --build
```

First run builds the images (a few minutes). It creates the database tables automatically. To confirm everything is up:

```
docker compose -f docker-compose.prod.yml ps
```

4. Let colleagues reach it (office LAN)

1. Find the server's LAN IP: run `ipconfig` and note the IPv4 address

(e.g. `192.168.1.50`).

2. Open Windows Firewall for inbound TCP **3000** (once):

```
netsh advfirewall firewall add rule name="GGPL Quote" dir=in action=allow protocol=TCP localip=192.168.1.50
```

3. Colleagues open `http://192.168.1.50:3000` in their browser.

(On the server itself: `http://localhost:3000`.)

A fixed IP (or DHCP reservation) for the server PC keeps the address stable.

5. First login

Login is enabled. Sign in as the seeded admin and then manage users in

Settings → Access / Users:

- `shashnam / shashnam` (admin) — **change this password immediately.**
- Seeded team logins: `sales/sales123`, `estimation/estimation123`, `technical/technical123`.

The granular 11-stage workflow is enabled by default in this compose.

6. Day-to-day

- **Stop:** `docker compose -f docker-compose.prod.yml down`
- **Start again:** `docker compose -f docker-compose.prod.yml up -d`

(containers also auto-start after a reboot — `restart: unless-stopped`.)

- **Update after code changes:** `docker compose -f docker-compose.prod.yml up -d --build`
- **Logs:** `docker compose -f docker-compose.prod.yml logs -f`

7. Back up the data

The database lives in the `postgres-data` Docker volume. Back it up regularly:

```
docker compose -f docker-compose.prod.yml exec postgres \
  pg_dump -U postgres gasket_quote &gt; backup_YYYYMMDD.sql
```

Restore into a fresh stack:

```
docker compose -f docker-compose.prod.yml exec -T postgres \
  psql -U postgres gasket_quote &lt; backup_YYYYMMDD.sql
```

B. Alternative: native (no Docker)

Use this only if Docker can't be installed. More moving parts.

On the server PC install: **Python 3.11**, **Node 20**, and **PostgreSQL 16**

(or reuse the JSON file store for a single machine — not recommended for a shared office setup, and avoid running it from a OneDrive-synced folder, which causes file-lock errors).

1. **Database:** create a Postgres DB `gasket_quote` and note its URL.
2. **API:**

```
pip install -r requirements.txt
pip install -e packages -e apps/api
set DATABASE_URL=postgresql://postgres:&lt;pw>&gt;@localhost:5432/gasket_quote
set AUTH_SECRET=&lt;long random>&gt;;
set ENABLE_GRANULAR_WORKFLOW=true
set OPENAI_API_KEY=sk-...
cd apps/api
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

3. Web (production build, not `next dev`):

```
cd apps/web
set NEXT_PUBLIC_ENABLE_GRANULAR_WORKFLOW=true
npm ci
npm run build
set API_BASE_URL=http://127.0.0.1:8000
npm run start -- -H 0.0.0.0 -p 3000
```

4. Open firewall port 3000 (step A.4) and share `http://<server-ip>:3000`.

5. To keep both running after logout/reboot, wrap them as Windows services

(e.g. with NSSM) or scheduled tasks.

Security notes

- Keep this on the LAN only — do **not** port-forward 3000 to the internet.
- Set a strong `AUTH_SECRET` and `POSTGRES_PASSWORD`; change the seeded admin

password on first login.

- Rotate the OpenAI key if it has been shared.
- `.env` is gitignored — never commit real secrets.